GoReview AI — Completion Plan

Project: GoReview AI — Open-source CLI architectural smell detector
Date: April 2026
Team: RAHIOUI · NAZIH · Machnaoui · Hattabi · HOUSSY

---

1. Current State Assessment

What Is Already Done

Phase	Component	Status
Phase 1	`core/` — Finding, Score, Report	✅ Complete
Phase 2	`languages/golang/` — AST parser + walker	✅ Complete
Phase 2	`languages/python/` — AST parser + walker	✅ Complete
Phase 3	Go detectors: god_struct, concrete_dep, complexity	✅ Complete
Phase 3	Python detectors: god_class, complexity	✅ Complete
Phase 4	`llm/` — Provider interface, context_builder	✅ Complete
Phase 4	LLM backends: Ollama, OpenAI	✅ Complete
Phase 5	`output/` — terminal, JSON, HTML	✅ Complete
Phase 6	`main.go` — full CLI (cobra, all flags, CI mode)	✅ Complete

The core v0.1 and most of v0.2 are **done**. The tool is functional today for Go and Python analysis with local or cloud LLM enrichment.

What Is Missing

Item	Version	Priority
Anthropic/Claude LLM provider	v0.2	High
Primitive obsession detector (Go)	v0.2	High
Primitive obsession detector (Python)	v0.2	Medium
Tree-sitter fallback (100+ languages)	v0.3	Medium
TypeScript/JS parser + detectors	v0.3	Medium
Shotgun surgery detector (git-based)	v1.0	Low
VS Code extension	v1.0	Low
AWS deployment	Production	High

---

2. Remaining Implementation — Detailed Plan

2.1 Anthropic / Claude Provider *(v0.2 — 1 day)*

File to create: `llm/anthropic/client.go`

The existing `llm.Provider` interface requires only one method: `Enrich(ctx, []Finding) ([]Finding, error)`. The Anthropic provider mirrors the OpenAI client structure.

```
llm/
├── provider.go          (existing interface)
├── context_builder.go   (existing — no changes needed)
├── ollama/client.go     (existing)
├── openai/client.go     (existing)
└── anthropic/client.go  ← NEW
```

Implementation steps:

1. Add the `anthropic-go` SDK to `go.mod`:

   ```
   go get github.com/anthropics/anthropic-sdk-go
   ```

2. Create `llm/anthropic/client.go` implementing `llm.Provider`:
   - Constructor: `New(apiKey, model string) llm.Provider`
   - Default model: `claude-haiku-4-5-20251001` (fast, cheap, good for structured JSON)
   - Use `BuildPrompt(f)` from `context_builder.go` — no changes needed
   - Parse the JSON response into `f.Explanation`, `f.RefactorBefore`, `f.RefactorAfter`, `f.Effort`

3. Wire into `main.go` `buildProvider()`:

   ```
   case "anthropic":
       apiKey := flagLLMAPIKey
       if apiKey == "" { apiKey = os.Getenv("ANTHROPIC_API_KEY") }
       return anthropicLLM.New(apiKey, flagLLMModel), nil
   ```

4. Add `--llm-provider anthropic` to the flag help text.

Recommended model for production use: `claude-haiku-4-5-20251001`
Fast (< 1s per finding), cheap ($0.25/M input tokens), outputs clean JSON reliably.

2.2 Primitive Obsession Detector — Go *(v0.2 — 1–2 days)*

File to create: `detectors/primitiveobs/detector.go` + `detector_test.go`

What it detects: Struct fields or function parameters that use raw primitives (`string`, `int`, `bool`) for domain concepts that should be typed (e.g., `UserID string`, `Price float64`, `Email string`).

Detection logic (AST-based):

```
For each struct in ParsedFile.Types:
  count fields with type string, int, float64, bool
  if count >= threshold (default: 3 primitive fields of similar domain):
    emit Finding{
```

```
      SmellType: "primitive_obsession",
      Pattern:   "Value Object",
      Severity:  core.Medium,
    }
```

Heuristics for domain detection:

- Field name ends in `ID`, `Id`, `Code`, `Name`, `Email`, `Phone`, `URL`, `Path` → strong signal
- Field name ends in `Count`, `Amount`, `Price`, `Total` → strong signal
- Threshold: ≥ 3 such fields in a single struct = finding

Integration: Add `primitiveobs.Detector{}` to the Go detector slice in `main.go`.

2.3 Primitive Obsession Detector — Python *(v0.2 — 1 day)*

File to create: `detectors/python/primitiveobs/detector.go` + `detector_test.go`

Same logic as the Go version but operating on `languages/python.ParsedFile`. Python `__init__` parameters typed as `str`, `int`, `float` with domain-semantic names trigger the finding. Add `primitiveobs.Detector{}` to the Python detector slice in `main.go`.

2.4 Tree-sitter Fallback *(v0.3 — 1 week)*

Purpose: Support Ruby, C++, Rust, Swift, PHP and any language supported by tree-sitter without writing a custom parser.

File to create: `languages/treesitter/parser.go`

Dependency:

```
go get github.com/smacker/go-tree-sitter
```

ParsedFile structure for tree-sitter:

```go
type ParsedFile struct {
    Path     string
    Language string
    Root     *sitter.Node   // raw AST node
    Source   []byte
}
```

Detectors for tree-sitter: Create a `detectors/generic/` package with heuristic detectors that operate on raw AST nodes (node counts, depth, function sizes). These run as a fallback when no native parser exists for the language.

Integration in `main.go`:

```go
// After Go and Python passes — tree-sitter fallback for other extensions
tsFiles, _ := treesitter.ParseDir(dir, alreadyParsedPaths)
if len(tsFiles) > 0 {
    allFindings = append(allFindings, generic.RunAll(ctx, tsFiles)...)
}
```

2.5 TypeScript / JavaScript Support *(v0.3 — 1 week)*

Files to create:

- `languages/typescript/parser.go` — uses tree-sitter-typescript grammar
- `detectors/typescript/godclass/detector.go`
- `detectors/typescript/complexity/detector.go`

TypeScript analysis via tree-sitter is the fastest path since the TypeScript Compiler API would require a Node.js subprocess. Using tree-sitter keeps everything native Go with no external runtime.

2.6 Shotgun Surgery Detector *(v1.0 — 3–5 days)*

What it detects: A change that forces modifications across more than 5 files — detected by mining git history.

Implementation:

```
// detectors/shotgunsurgery/detector.go
// Run: git log --format="%H" -n 200
// For each commit: git diff-tree --no-commit-id -r --name-only <hash>
// Build co-change matrix: file pairs that change together
// Flag any file that appears in > threshold co-change clusters
```

Dependencies: Only `os/exec` to call git — no new Go dependencies.

This detector requires a git repository. Skip gracefully if `.git` is not present.

---

3. Parsing Strategy

Go — Native AST (stdlib)

Go analysis uses `go/parser`, `go/types`, `go/ast` from the standard library. No external dependency.

```
ParseDir(dir) → []ParsedFile{
    AST:   *ast.File       // full syntax tree
    Pkg:   *types.Package  // resolved type info
    Fset:  *token.FileSet  // position mapping
}
```

Why native: Go's stdlib parser is the most accurate option for Go code. It resolves types (needed for `concrete_dep` — checking if a field is an interface or a concrete struct), handles generics (Go 1.18+), and has zero latency.

Python — `go-ast-python` / regex hybrid

Python analysis uses a pure-Go Python AST parser. The current `languages/python/parser.go` walks `.py` files and extracts class definitions, method counts, and cyclomatic complexity via line-by-line analysis.

Key parsing approach:

- Use `regexp` to identify `class`, `def`, `if`, `for`, `while`, `and`, `or` keywords
- Count nesting depth via indentation tracking

- No Python interpreter required — 100% offline

Limitation: No type resolution. Primitive obsession detection for Python is purely name-based (parameter names ending in `_id`, `_email`, `_price`, etc.).

Tree-sitter — Universal Fallback

For all other languages, `go-tree-sitter` parses source into a concrete syntax tree. Detectors query the tree using S-expression patterns:

```
// Find all function definitions with bodies > 50 lines
query := "(function_definition body: (_) @body)"
```

This approach works identically for Ruby, Rust, C++, PHP, Swift without language-specific code.

What NOT to do

- Do **not** shell out to `ast` module in Python — it would require a Python interpreter at runtime
- Do **not** use regex on Go source — the native AST is always available and more reliable
- Do **not** send raw source to the LLM — the `context_builder.go` correctly sends only smell summaries (component name, metrics, location)

---

4. LLM Strategy

Local LLM — Recommended Models (Ollama)

Model	Size	Code quality	Speed	Use case
`qwen2.5-coder:7b`	4.7 GB	⭐⭐⭐⭐⭐	Fast	**Best default for gorview**
`deepseek-coder-v2:16b`	9.1 GB	⭐⭐⭐⭐⭐	Medium	Best quality, needs 16 GB RAM
`codestral:22b`	12 GB	⭐⭐⭐⭐	Slow	High-RAM machines only
`llama3.2:3b`	2 GB	⭐⭐⭐	Very fast	Low-RAM / quick mode

Recommendation: **`qwen2.5-coder:7b`**
It produces structured JSON reliably, understands Go and Python architecture patterns, runs in under 2 seconds per finding on a mid-range laptop (8 GB RAM), and is free.

Setup:

```
ollama pull qwen2.5-coder:7b
gorview ./... --llm --llm-model qwen2.5-coder:7b
```

Cloud LLM — Provider Comparison

Provider	Model	Cost / 1000 findings	Latency	Quality
Anthropic	`claude-haiku-4-5`	~$0.05	< 1s	⭐⭐⭐⭐⭐
OpenAI	`gpt-4o-mini`	~$0.10	< 1s	⭐⭐⭐⭐

OpenAI	`gpt-4o`	~$2.50	2–3s	⭐⭐⭐⭐⭐

Recommendation: `claude-haiku-4-5-20251001` for cloud usage.
Produces valid JSON output more consistently than GPT-4o-mini, costs half as much, and is faster. The structured JSON output is critical because `gorview` parses `explanation`, `refactor_before`, `refactor_after`, and `effort` from each response.

Privacy Guarantee (from PDF spec)

Only the following is sent to the LLM — **never raw source code:**

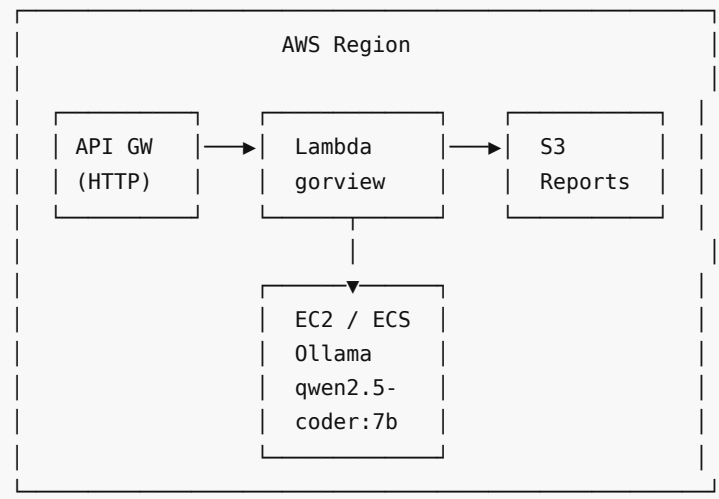
```
smell_type:  god_struct
component:   UserService
location:    service/user.go:14
metrics:     fields=19, methods=28
pattern:     Façade
```

---

5. AWS Deployment

The PDF describes gorview as a local CLI tool. AWS deployment adds a **server mode** so teams can run gorview as a centralized service (CI webhooks, web dashboard, shared Ollama instance).

Architecture



Component Details

Lambda — gorview Analysis Function

- **Runtime:** `provided.al2023` (Go binary, no runtime overhead)
- **Trigger:** API Gateway POST `/analyze` with JSON body `{"repo_url": "...", "format": "json"}`
- **What it does:**
  1. Clone repo into `/tmp` (Lambda ephemeral storage, up to 10 GB)
  2. Run `gorview` static analysis (no LLM, fast)
  3. Upload JSON report to S3

4. Return S3 pre-signed URL

- **Timeout:** 5 minutes
- **Memory:** 512 MB
- **Cost:** ~$0.00001 per analysis (essentially free)

EC2 — Ollama Instance (optional)

For teams wanting LLM enrichment without sending code to OpenAI/Anthropic:

- **Instance type:** `g4dn.xlarge` (1x T4 GPU, 16 GB RAM) — ~$0.52/hr spot
- **Or:** `m7i.large` (CPU-only, 8 GB RAM) — ~$0.096/hr, runs `qwen2.5-coder:7b` in ~3s/finding
- **Ollama runs as a service** on port 11434 (internal VPC only, never public)
- Lambda calls it via VPC: `--llm-base-url http://<ec2-private-ip>:11434`

Alternative: Use AWS Bedrock (no EC2 needed)

Instead of running Ollama on EC2, use **AWS Bedrock** with Claude Haiku:

```
// llm/bedrock/client.go (new file)
// Uses AWS SDK v2: github.com/aws/aws-sdk-go-v2/service/bedrockruntime
// Model ID: anthropic.claude-haiku-4-5-20251001-v1:0
// No Ollama server needed, pay-per-token, serverless
```

This is the recommended production path: Lambda for analysis + Bedrock for LLM enrichment = fully serverless, zero servers to manage.

S3 — Report Storage

- Bucket: `gorview-reports-<account-id>`
- Object key: `reports/<timestamp>/<repo-slug>/report.json`
- Pre-signed URL valid for 1 hour
- Lifecycle: auto-delete after 30 days

Deployment Steps

1. **Build the Lambda binary:**

```
GOOS=linux GOARCH=amd64 go build -o bootstrap .
zip gorview-lambda.zip bootstrap
```

2. **Create Lambda function:**

```
aws lambda create-function \
  --function-name gorview-analyze \
  --runtime provided.al2023 \
  --handler bootstrap \
  --zip-file fileb://gorview-lambda.zip \
  --role arn:aws:iam::<account>:role/gorview-lambda-role \
  --timeout 300 \
  --memory-size 512
```

3. **Add API Gateway trigger** (HTTP API, POST /analyze)

4. **Set environment variables in Lambda:**

```
ANTHROPIC_API_KEY=sk-ant-...   # if using Anthropic
GORVIEW_REPORT_BUCKET=gorview-reports-<account-id>
```

5. **IAM permissions needed:**
   - `s3:PutObject` on the reports bucket
   - `bedrock:InvokeModel` if using Bedrock
   - `logs:CreateLogGroup`, `logs:PutLogEvents`

Lambda Handler Wrapper

Add a `cmd/lambda/main.go` entry point that wraps the existing analysis logic:

```go
package main

import (
    "github.com/aws/aws-lambda-go/lambda"
    // ...gorview imports
)

type Request struct {
    RepoURL string `json:"repo_url"`
    Format  string `json:"format"`
    LLM     bool   `json:"llm"`
}

type Response struct {
    ReportURL string `json:"report_url"`
    Score     int    `json:"score"`
}

func handler(ctx context.Context, req Request) (Response, error) {
    // clone repo to /tmp, run analysis, upload to S3
}

func main() { lambda.Start(handler) }
```

---

6. Implementation Order (Remaining Work)

```
Week 1 (v0.2 complete):
  Day 1:  llm/anthropic/client.go + wire into main.go
  Day 2:  detectors/primitiveobs/detector.go (Go)
  Day 3:  detectors/python/primitiveobs/detector.go (Python)
  Day 4:  Integration testing — go build ./... && go test ./...
  Day 5:  AWS Lambda wrapper (cmd/lambda/main.go)

Week 2 (v0.3 start):
  Day 1–3: languages/treesitter/parser.go + go-tree-sitter dep
  Day 4–5: detectors/generic/ (tree-sitter heuristic detectors)
```

```
Week 3–4 (v0.3 complete):
  TypeScript parser + detectors
  CI/CD pipeline (GitHub Actions → Lambda deploy)

Week 5–6 (v1.0 start):
  detectors/shotgunsurgery/ (git log analysis)
  Plugin API stabilization
  VS Code extension scaffold
```

---

7. File Checklist — Remaining Files to Create

File	Purpose	When
`llm/anthropic/client.go`	Claude Haiku/Sonnet provider	Week 1, Day 1
`detectors/primitiveobs/detector.go`	Primitive obsession for Go	Week 1, Day 2
`detectors/primitiveobs/detector_test.go`	Tests	Week 1, Day 2
`detectors/python/primitiveobs/detector.go`	Primitive obsession for Python	Week 1, Day 3
`cmd/lambda/main.go`	AWS Lambda entry point	Week 1, Day 5
`languages/treesitter/parser.go`	Universal language fallback	Week 2
`detectors/generic/complexity/detector.go`	Tree-sitter complexity	Week 2
`languages/typescript/parser.go`	TypeScript via tree-sitter	Week 3
`detectors/typescript/godclass/detector.go`	TS god class	Week 3
`detectors/typescript/complexity/detector.go`	TS complexity	Week 3
`detectors/shotgunsurgery/detector.go`	Git co-change analysis	Week 5

---

8. Summary

The project is approximately **75% complete**. The core pipeline (parse → detect → score → output) is fully functional for Go and Python. The CLI with all flags (including CI mode `--min-score`) is done.

The **3 highest-priority remaining items** to reach a shippable v0.2:

1. **Anthropic provider** — 1 file, 1 day, enables Claude as a backend
2. **Primitive obsession detector (Go)** — 1 file, 2 days, completes the detector table from the PDF
3. **AWS Lambda wrapper** — 1 file, 1 day, enables team/CI deployment

After those three, gorview v0.2 is complete and matches every feature listed in the PDF specification.

GoReview AI · Projet d'Innovation · 2025–2026
RAHIOUI · NAZIH · Machnaoui · Hattabi · HOUSSY